Unit-8: Simulation of Computer System

Why simulations?

Simulation is a very important means in the field of power electronics, mechanical engineering, mathematical formulations, and field of science and research etc.

Simulation leads to

- Saving of development time,
- Saving of costs ('burnt power circuits tend to be expensive'),
- Better understanding of the function,
- Testing and finding of critical states and regions of operation,
- Fast optimization of system and control.

Today it is difficult to imagine the task of power electronics development without the help of simulation.

Simulation Tools:

A simulation tool is a software application or platform used to create a virtual model of a system, process, or environment for the purpose of analysis, experimentation, and training. These tools allow users to explore and understand complex systems by simulating their behavior under various conditions without having to interact with the real system, which can be costly, time-consuming, or dangerous.

When someone is learning how to drive a car, the driving school teachers train them on a simulator before teaching them how to drive an actual car on the road. Knowing or testing a product or machine's behavior before manufacturing or using it is always good. You can know how the product will look and behave, and if it isn't working according to your expectations, you can always make changes. When the cost of improper operations is high, a mock-up of the actual control panel is connected to real-time simulator tools.

Simulation tools are used to understand and design equipment that will be close to the desired outcome. Simulation tools help in predicting the behavior of the system. You can utilize simulation tools to create and evaluate new designs, spot issues in the existing designs and test a system under certain conditions.

For example… electronic simulation tools are used to replicate the actual electronic device. Similarly mathematical simulation tools show how mathematical equations works.

Properties of good simulation tools:

Some properties of a good tool are

- Comfortable
- Correct models for the elements (as simple as possible but, as good as necessary)
- Correct error messages
- Robust (*powerful/strong*) execution of the simulation.
- Sophisticated integration algorithm (various algorithms to be chosen for the type of model)
- Good output of the simulation results (formats which can be exported to other programs)
- Support from the manufacturer
- Portability of models from one program version to succeeding one

Different Simulation Tools:

Simulation tools are used in various industries, and they are highly beneficial. There are a number of powerful simulation tools available. All of them have advantages and disadvantages.

Following are popular simulation tools available:

Matlab / Simulink / SimPowerSystems:

Matlab is a mathematical tool that has been established for a long time. Toolboxes for various applications exist. One of them is Simulink, a graphical tool for the entering of functions. Simulink itself can be expanded with another toolbox: SimPowerSystem. This toolbox is designed for the simulation of electrical power systems including power electronics. The elements of the various toolboxes can be combined.

PSpice:

PSpice (Personal Simulation Program with Integrated Circuit Emphasis) is a software tool used for simulating the behavior of analog and mixed-signal circuits. It is widely used in electrical engineering for circuit design, analysis, and testing.

It started as a simulation tool for low-power electronic circuits. A large library with PSpice models for various electronic components exists. Further models can be added. Today it can simulate analog and digital circuits with a lot of features. The representation of numerical blocks and controllers is difficult.

AnyLogic:

AnyLogic is the first simulation tool to introduce multi-method simulation modeling. The tool can convert flowcharts into interactive movies with 2D and 3D graphics. Users can present their models to their stakeholders in a self-explanatory and visually attractive manner. The tool provides an extensive set of graphical objects to visualize equipment, staff, vehicles, buildings, and many more items.

AutoDesk:

AutoDesk is a simulation tool that lets users test their designs efficiently to ensure they survive in the real world. Users can simulate their product digitally by reducing the cost of prototyping. The tool provides a feature named Fusion 360, a cloud simulation tool. It is an integrated CAD, CAM, and CAE tool that provides a range of simulation capabilities, It eliminates the need for any hardware.

Users can test for 8 different criteria, which include events, nonlinear and more. The tool lets users remove unnecessary bulk from the existing design. Users can achieve advanced design refinements by using the 3D mesh outputs as their guide.

Ansys:

Ansys, as a simulation tool, provides more time to optimize and innovate product performance. Users can create advanced physics models and analyze various fluid phenomena. The tool provides an intuitive and customizable space.

Users can accelerate their design cycle with the tool. The tool provides the best physics models and efficiently solves complex models. Users can accelerate their

pre and post-processing as the tool provides a streamlined workflow for application and meshing setups.

Arena Simulation:

Arena Simulation provides discrete event simulation. It allows users to quickly analyze a system's behavior or process over time. Users can use the flowchart modeling methodology. It provides a vast library of pre-defined building blocks to model the process without custom programming. The tool lets users complete a range of statistical distribution options to accurately model process variability.

The tool has the ability to define routes and paths for simulation. It provides statistical reports and analysis. Users can see realistic 3D and 2D animations to visualize results beyond numbers.

Incontrol:

Incontrol is a simulation tool that helps users cope with costs, time, resources, reliability, productivity, sustainability, and safety. The tool provides two software- Supply Chain Simulation and Crowd Management Simulation software.

The supply chain simulation software is a comprehensive platform that offers users a fully scalable, easy-to-use, and configurable simulation environment. With data-driven answers, it helps users solve complex processes, people, infrastructure, and technology-related challenges. Users can virtually test and improve scenarios through the system lifecycle without disturbing the actual system.

The crowd management simulation is designed for the execution and creation of large crowds simulation in complex infrastructures. The users can use the tool to evaluate the safety and performance of the environment.

Introduction to Simulation Language:

A simulation language is a special purpose language structured to meet the programming requirements of the simulation models of a specific class of situations. The analyst develops the simulation model, employing this special purpose language, which is applicable over a class of applications.

In the sense, these language are comparable to FORTAN, C#, VB.NET, or Java but also include specific features to facilitate the modeling process.

Example:

General Purpose Simulation System(GPSS)

GPSS is a simulation programming language developed by IBM for modeling and simulating discrete event systems such as:

- Queueing systems (banks, hospitals)
- Manufacturing systems
- Inventory systems
- Traffic systems
- Computer networks

It is mainly used to simulate real-world systems where events occur at specific points in time. Similarly SIMAN (**SIM**ulation **AN**alysis) and SLAM II are more appropriate for simulating the manufacturing and material handling systems. Some other modern simulation languages are GPSS/H, GPSS/PC, SLX and SIMSCRIPT III.

Features of Simulation Language:

The important features of simulation language can be identified as follows:

- Generation of large streams of random numbers
- Generation of random variates from a large number of probability distributions
- Determining the length of simulation run and length of wrapping (covering) up period
- Advancing the simulation clock
- Scanning the event list to determine the next earliest event to occur

- Collecting data
- Analyzing data and setting confidence intervals

Objectives of Simulation Language:

Simulation software packages are designed to meet the following objectives:

- To conveniently describe the elements, which commonly appear in simulation, such as the generation of random deviates.
- Flexibility of changing the design configuration of the system so as to consider alternate configuration.
- Internal timing and control mechanism, for book keeping of the vital information during the simulation run.
- To obtain conveniently, the data and statistics on the behavior of the system.

Advantage of Simulation Language:

- **Less programming time and effort:** Since most of the features to be programmed are in-built, simulation languages take comparatively *less programming time and effort.*
- **Provide a natural framework for simulation modelling:** Since simulation language consists of blocks, specially constructed to simulate the common features, they *provide a natural framework for simulation modelling*.
- **Easily be changed and modified:** The simulation models coded in simulation language can *easily be changed and modified*.
- **Error detection and analysis:** The *error detection and analysis* is done automatically in simulation languages.
- **Easy to use:** The simulation models developed in simulation languages, especially the specific application packages, called simulators, are very *easy to use*.

Simulation Language- GPSS:

GPSS, which stands for ***General Purpose Simulation System***. This language was developed primarily by Geoffrey Gordon at IBM around 1960, and has contributed important concepts to every commercial discrete event Computer Simulation Language developed ever since.

GPSS (General Purpose Simulation System) is a simulation language designed for modeling discrete event systems. It was developed to model and analyze systems where the state changes at discrete points in time, such as manufacturing processes, computer networks, and service systems like banks or hospitals.

GPSS is a discrete time simulation general-purpose programming language, where a simulation clock advances in discrete steps. A system is modeled as transactions enter the system and are passed from one service (represented by blocs) to another. That is, in GPSS, entities (such as customers, machine parts, or vehicles) are modeled as transactions that move through a series of service blocks. User translates his problem into a conceptual model, which is a block diagram. Then GPSS software package, processes this block diagram, Executes the simulation run, and Produces statistics.

GPSS was designed especially for analysis who were not necessarily computer programmer. It is particularly suited for modeling traffic and queuing systems. A **GPSS** programmer does not write a program in the same sense as SIMSCRIPT but, he construct a block diagram – a network of interconnected blocks, each performing a special simulation oriented function.

GPSS provides a set of 48 different blocks to choose from each of which can be used repeatedly. Each block has a name and specific task to perform. Moving through the system of one block to another block are called transaction and entities are customer, messenger, machine parts, vehicle, etc.

Basic Structure

A transaction is a GPSS object with a number of attributes. A transaction is like a customer entering into the process for service. It is a dynamic entity that represents an item flowing through the system being simulated.

It can represent:

- A customer in a bank
- A patient in a hospital
- A job in a CPU
- A car in a service station
- A packet in a network

A single transaction may represent several individual entities. Each transaction has to be generated either one at a time or in batches. Once the transaction are appear into the system, they must be contained exactly in one action Block. However, a Block may contain many transactions.

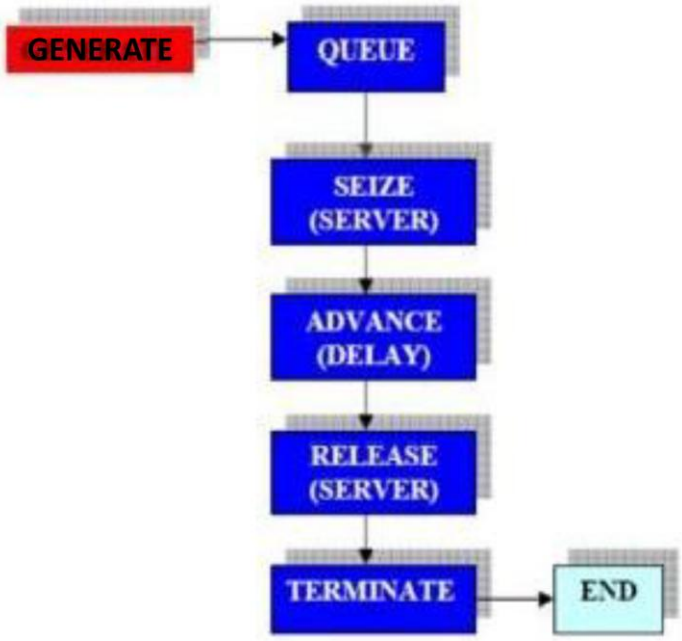


Fig: Basic Structure GPSS

- A **GENERATE** Block generates a stream of transactions with a specific set of behavior that is it create entities (transactions) at specified intervals.

No transaction may again enter this block. Behavior could be deterministic, stochastic, functional, etc. A Transaction leaving a

GENERATE Block descends into the next available Block it finds. The entering Block shouldn't deny entries to transactions. Otherwise, system backups may result.

- A **QUEUE** Block manage the entities waiting in line and their departure. A **QUEUE** Block never ***refuses any transaction***. If a transaction cannot enter into the next Block, it stays at the current Block. Therefore, a QUEUE simulates an infinitely long buffer.

- **SEIZE** Block Allocates a resource to an entity. A transaction attempts to **SEIZE** ***a facility (server, router, CPU) for service.*** If it succeeds, it would leave the current Block and start using the facility. If not, it stays where it is until the next time. As long as a facility is occupied, it cannot allow another transaction to SEIZE it.
- **ADVANCE Block** delays an entity for a specified amount of time. It captures the transaction and imposes a ***delay on it wherever it is***. The delay could be deterministic, probabilistic, etc.
- A **RELEASE** Block forces a transaction to release its facility i.e. it frees a resource from an entity. For every successful SEIZE, there must be a RELEASE.
- A **TERMINATE** Block ***kills the entering transaction*** here.

Hence…

If transaction is moving object (like customer), Then block is the action or step that the customer performs.

Example (Bank System): Suppose we simulate a bank:

Block	**Meaning**
GENERATE	Create customer
QUEUE	Customer waits in line
SEIZE	Customer takes counter
ADVANCE	Service time
RELEASE	Leave counter
TERMINATE	Exit system

GPSS - Basic Commands

- **START:** set the termination count and begin a simulation
- **EXIT:** end the GPSS world session
- **CLEAR:** reset statistics and remove transactions
- **CONTINUE:** resume the simulation
- **HALT:** stop the simulation and delete all queued commands
- **INCLUDE:** read and translate a secondary model file
- **INTEGRATE:** automatically integrate a time differential in a use variable
- **REPORT:** set the name of the report file or request an immediate report
- **RESET:** reset the statistics of the simulation
- **SHOW:** evaluate and display expression
- **STEP:** attempt a limited number of block entities
- **STOP:** set a stop condition based on block entry attempts
- **STORAGE:** define a storage entity
- **TABLE:** define a table entity
- **VARIABLE:** define a variable entity

Transaction:

In General Purpose Simulation System (GPSS), a transaction is a dynamic entity that represents the flow of items through a simulation model. Transactions are essentially the objects or items being processed within the system. They move through various blocks or components, experiencing changes in state and interacting with resources according to the logic defined in the simulation model.

Transaction in GPSS is a process that represents the real-world system you are modeling which is executed by moving from block to block. Each transaction in the model is contained in exactly one block, but one block may contain many transactions.

The Various Blocks of a GPSS are as follows:

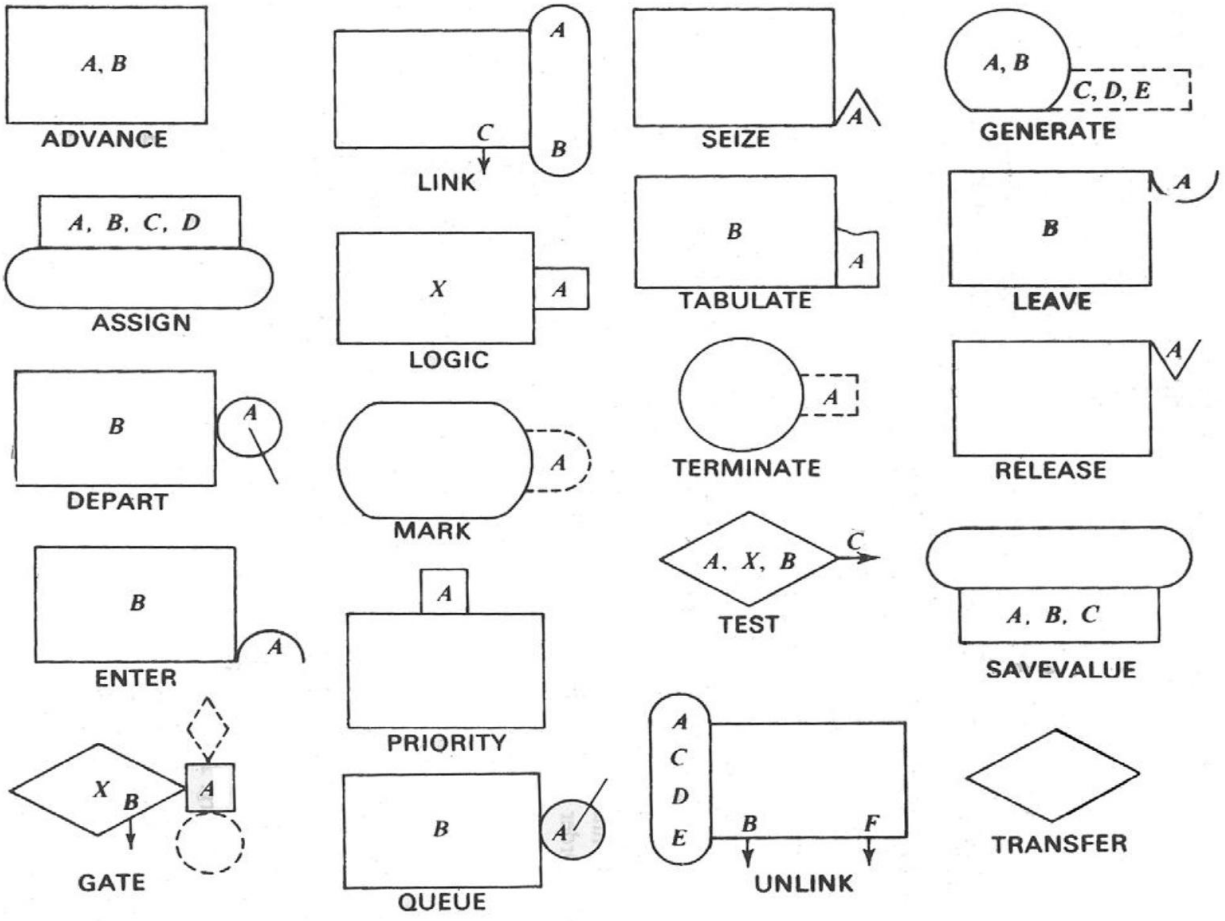
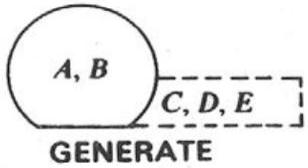


Fig: Different Blocks used in GPSS

1. GENERATE BLOCK:

This block will produce or create a transactions for future entry into the simulation. It creates with inter-arrival times determined by the attribute values. The Label is optional. The distribution of inter-arrival times follows a uniform probability distribution.



A - Average value of uniform distribution (mean)

B - Inter generation time half-range or Function Modifier

C - Start delay before first transaction is generated

D – Maximum number of transaction generated

E - Priority allocated to transaction. Zero is the default

Syntax:
Line Number Label GENERATE A, B, C, D, E (*Line Number and Label Optional)*

Example-1: GENERATE 5, 2

In this example:

- **A (5)**: The average inter-arrival time between transactions is 5-time units.
- **B (2)**: The half-range for the inter-arrival times is 2-time units, which means the actual inter-arrival times will be uniformly distributed between 5−2 and 5+2, i.e., between 3 and 7 time units.

Inter-Arrival Time Calculation

The inter-arrival time for each transaction will be a random value uniformly distributed between: A−B and A+B
So, in this case:
5−2=3 and
5+2=7
Therefore, transactions will be generated with inter-arrival times randomly selected between 3 and 7 time units.

Example-2:

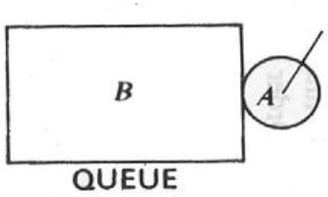
10 GENERATE 5, 2, 0, 100, 1

- This line would create a GENERATE block on line 10 with:
- An average inter-arrival time of 5 time units.
- A half-range of 2 time units (creating variability between 3 and 7 time units).
- No start delay (0).
- A maximum of 100 transactions generated.

- A priority of 1 for the transactions.

2. QUEUE BLOCK:

The QUEUE block in GPSS is used to place entities into a specified queue. This block will instruct GPSS to start gathering queuing statistics on the queue named in its attribute value. The Label is optional but may be necessary if you have to refer to this line from somewhere else in the program.



Syntax:

Line Number Label QUEUE A

Parameters

- **Line Number**: Optional. The line number in the GPSS code for reference.
- **Label**: Optional. A user-defined name for the block for easier reference.
- *A = Name of queue (For Example: Garage)*

Example: QUEUE Garage

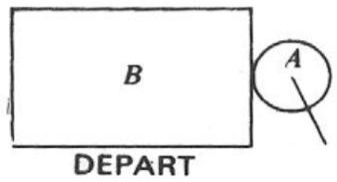
Here,

- The QUEUE block places entities into a queue named Garage.

3. DEPART BLOCK:

The DEPART block in GPSS is used to remove entities from a specified queue. This block decrements the queue length by 1, indicating that an entity has left the queue. It is typically used in conjunction with the QUEUE block to manage entities entering and exiting queues.

This block instructs GPSS that a transaction is leaving the queue named in its attribute value. This is necessary in order to compile the statistics on the queue. The Label is optional.



Syntax:

Line Number Label DEPART A

- **A**: Name of the queue (a user-defined identifier for the queue).

Parameters

- **Line Number**: Optional. The line number in the GPSS code for reference.
- **Label**: Optional. A user-defined name for the block for easier reference.

Example: DEPART Garage

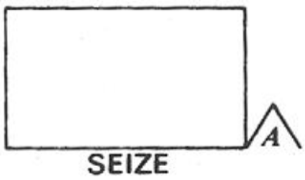
Here, The DEPART block removes entities from the queue named Garage.

Note: *If* ***QUEUE*** *block is used in the model then the corresponding* ***DEPART*** *block should follow it.*

4. SEIZE BLOCK:

The SEIZE block in GPSS is used to allocate a facility (resource) to an entity. This block ensures that an entity can only proceed if the specified facility is available. If the facility is busy, the entity must wait until it becomes available.

This block allows the transaction to seize (*acquire*) a **Facility** if it is free. Thus it may be a car "seizing" a "facility" such as a petrol pump or a customer in a supermarket "seizing" a "facility" such as the checkout assistant. When the car or customer is being serviced by the facility, then it is said to "own the facility". The Label is optional.



Syntax:

Line Number Label SEIZE A

- **A**: Name of the facility (a user-defined identifier for the facility).

Parameters

- **Line Number**: Optional. The line number in the GPSS code for reference.
- **Label**: Optional. A user-defined name for the block for easier reference.

Example: SEIZE PUMP

Here, The SEIZE block attempts to allocate the facility named PUMP to an entity.

Note: *A transaction can only seize a facility if it is free or else wait until the owning transaction releases it.*

5. RELEASE BLOCK:

The RELEASE block in GPSS is used to free a previously seized facility (resource), making it available for other entities. This block is typically used after an entity has completed its use of the facility, allowing the facility to be allocated to the next entity waiting in the queue.

A transaction entering this block informs GPSS that it is giving up ownership of the **Facility** named in its attribute value. The Label is optional.

Syntax:

Line Number Label RELEASE A

A = Name of Facility (For Example: Pump)

Parameters

- **Line Number**: Optional. The line number in the GPSS code for reference.
- **Label**: Optional. A user-defined name for the block for easier reference.

Example: RELEASE PUMP

Here,

The RELEASE block frees the facility named PUMP, making it available for the next entity.

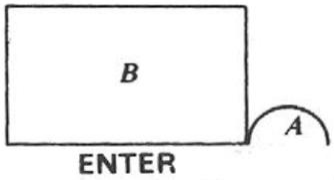
Note: *By giving up ownership of the facility, the transaction makes it available for another transaction that may be waiting to use it. If* ***SEIZE*** *block is used in the model then the corresponding* ***RELEASE*** *block should follow* it.

6. ENTER BLOCK:

This Block instructs GPSS that a transaction has entered STORAGE.

- The first attribute value gives the name of storage (Storage Contains Multiple Service Points)
- The second attribute value gives the amount the storage will be incremented by.

When the transaction enters the ENTER block. STORAGE must be declared at the beginning of a program.



Syntax:

Line Number Label ENTER A, B

A = name of the storage (for Example: warehouse)

B = increment storage by this value

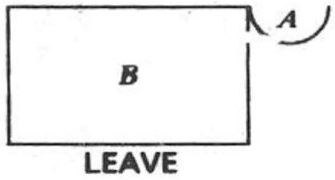
Example:

ENTER Warehouse, 25

In this example the Active Transaction demands 25 storage units from the storage units available at the Storage Entity named Warehouse. If there are not enough storage units remaining in the Storage Entity, the Transaction comes to rest on the Delay Chain of the Storage Entity.

7. LEAVE BLOCK:

This block instructs GPSS that a transaction is leaving a STORAGE. The first attribute gives the name of the STORAGE and the second attribute decrements the storage by the value of the attribute.



Syntax:

Line Number Label LEAVE A, B

A = name of the storage (for Example: warehouse)

B = Decrement storage by the value

Note: *If **ENTER** block is used in the model then the corresponding **LEAVE** block should follow it.*

Example:

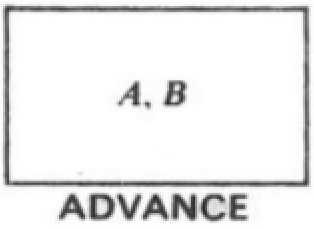
LEAVE Warehouse, 25

In this example, when a Transaction enters the LEAVE Block, the available storage units at the Storage Entity named Warehouse is decrement by 25 and total storage is decremented by 25 units.

8. ADVANCE BLOCK:

The ADVANCE block is used to model the time delay associated with a transaction as it undergoes a process or waits for a specific period. This block essentially makes the transaction wait for a defined amount of time, simulating the duration of a process, a delay, or a service time.

The servicing times follow a uniform probability distribution. The Label is optional.



Syntax:

Line Number Label ADVANCE A, B

A: The mean time delay (average value of uniform distribution (mean))

B: The standard deviation of the time delay (optional).

A transaction entering this block will be delayed by a time interval chosen at random from the specified probability distribution. If the second parameter (B) is omitted, the delay is deterministic (constant) and equal to A. If B is provided, the delay is stochastic (random) and follows a normal distribution with mean A and standard deviation B.

Example

To represent a process that takes exactly 10 time units, you can use:

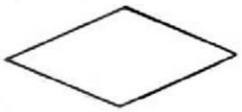
- ADVANCE 10

For a process that takes an average of 10 time units with a standard deviation of 2 time units, you can use:

- ADVANCE 10, 2

9. TRANSFER BLOCK:

The TRANSFER block is used to route transactions to different parts of the simulation model based on specific conditions or probabilities. It effectively controls the flow of transactions within the simulation, allowing for branching and decision-making.



Syntax

The TRANSFER block can have different forms depending on the type of routing required:

1. **Deterministic Transfer**
   TRANSFER A

2. **Probabilistic Transfer**
   TRANSFER A, B, C

3. **Condition-Based Transfer**
   TRANSFER IF <condition> , A, B

Examples

1. Deterministic Transfer

Transfers the transaction to a specified block unconditionally.

TRANSFER 20

In this example, the transaction will always transfer to block 20.

2. Probabilistic Transfer

Transfers the transaction to one of two specified blocks based on a probability.

TRANSFER 0.3,0.5,0.2

This transfers the transaction to one of three possible destinations based on the probabilities:

- 30% chance to the first specified block.

- 50% chance to the second specified block.
- 20% chance to the third specified block.

3. Condition-Based Transfer

Transfers the transaction based on a specified condition.

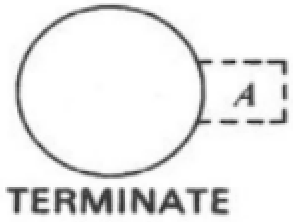
TRANSFER IF X1 > 5 , 15, 25

In this example, if the attribute X1 of the transaction is greater than 5, it will be transferred to block 15; otherwise, it will be transferred to block 25.

10. TERMINATE BOLOCK:

This block destroys any transaction entering it and removes it from computer memory. Each time a transaction enters this block it decrements a counter by an amount equal to its attribute value. The counter is set by the user upon starting the simulation.

In another words the TERMINATE block is used to end the life of a transaction within the simulation. When a transaction encounters a TERMINATE block, it is removed from the system. This block is essential for accurately modeling the completion of processes and the disposal of entities.



Syntax:

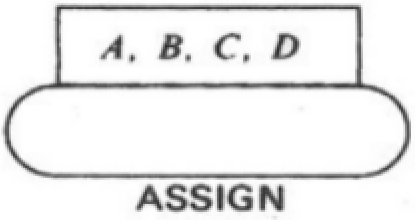
Line Number Label TERMINATE A

A = decrements simulation counter by this amount. The number of transactions to terminate. If omitted, the default is 1.

When the counter, set at the beginning of the simulation, reaches zero then the simulation is complete and a statistical report is produced on the outcome of the simulation.

11. ASIGN BLOCK:

The ASSIGN block is used to set or modify the value of a transaction's attribute. This block is essential for assigning specific values to attributes, which can then be used for decision-making, routing, and other operations within the simulation model.



Syntax

ASSIGN A, B, C, D

- A: The attribute number or name to be modified.
- B: The value to be assigned to the attribute.
- C: The condition under which the assignment is made.
- D: The action taken if the condition is not met.

Parameters

- A: This specifies the attribute being modified. It can be a predefined attribute number or a user-defined attribute name.
- B: This is the value that will be assigned to the attribute. The value can be a constant, a variable, or an expression.
- C: This is a condition that determines whether the assignment is made. It can be an expression involving attributes, constants, or other variables.
- D: This is the action taken if the condition specified in C is not met. It can involve transferring the transaction to a different block or setting a different value.

Examples-1:

ASSIGN 2000, 150.6

This is the simplest way to use the ASSIGN Block. The value 150.6 is assigned to Parameter number 2000 of the entering Transaction. If no such Parameter exists, it is created.

Example-2:
Consider a scenario where we want to assign a priority based on a condition:
ASSIGN Priority, 5, (CustomerType = 1), 10

In this example:

- Priority: The attribute being modified.
- 5: The value assigned if the condition is true.
- (CustomerType = 1): The condition under which the assignment is made.
- 10: The value assigned if the condition is false.

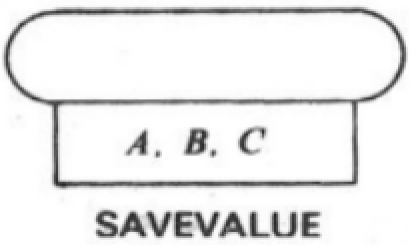
If the CustomerType attribute of the transaction is equal to 1, the Priority is set to 5. Otherwise, the Priority is set to 10.

12. SAVEVALUE BLOCK:

A SAVEVALUE Block changes the value of a Savevalue Entity. The SAVEVALUE block is used to:

- Store a value
- Add to a value
- Subtract from a value
- Maintain counters

It saves values in something called a Savevalue (SV).



Syntax:

SAVEVALUE A, B ,C

A - Savevalue Entity number. It may be followed by + or - to indicate addition or subtraction to existing value.

B - The value to be stored, added, or subtracted.

C – Optional Operand

- It is optional.
- Used in advanced GPSS versions.
- Usually used for indexing or additional control.
- In most basic GPSS problems, **C is not used**.

Example 1:

SAVEVALUE Account, 99.95

In this example, the Savevalue Entity named Account takes on the value 99.95.

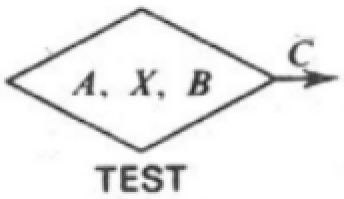
Example 2:

SAVEVALUE Kantipur, "The old name of Kathmandu"

In this example, the Savevalue Entity named Kantipur is assigned a string "The old name of Kathmandu". If the Savevalue Entity does not exist, it is created.

13. TEST BLOCK:

A TEST Block compares values, normally SNAs(Symbolic Name Addresses), and controls the destination of the Active Transaction based on the result of the comparison.



Syntax:

TEST O A, B, C
O - Relational operator. Relationship of Operand A to Operand B for a successful test. Required. The operator must be E, G, GE, L, LE, or NE (E=Equal, G=Greater than, GE=Greater than or equal, L=Less than, LE=Less than or equal, NE=Not equal)
A - Test value. Required.
B - Reference value. Required.
C - Destination Block number. Optional.

Note: In GPSS, Symbolic Name Addresses (SNAs) are used to refer to specific locations or values in the simulation model. They serve as labels or identifiers for variables, blocks, or other elements within the model. SNAs make the model more readable and easier to manage by allowing the use of meaningful names instead of numerical addresses.

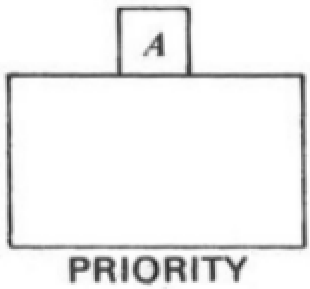
Example:

TEST G C1, 70000

In this example of a "Refuse Mode" TEST Block, the Active Transaction enters the TEST Block if the relative system clock value is greater than 70000. Otherwise, the Transaction is blocked until the test is true.

14. PRIORITY BLOCK:

A PRIORITY Block sets the priority of the Active Transaction.



Syntax:

PRIORITY A,B

Where:

- A is the operation (such as an addition or subtraction).
- B is the priority value or an expression that determines the new priority of the transaction.

Operations

The operation A can be:

- = (Set the priority to the value B)
- + (Increase the current priority by B)
- - (Decrease the current priority by B)

Example Usages

- Setting the Priority Directly

  PRIORITY =,10
  This sets the priority of the transaction to 10.

- Increasing the Priority

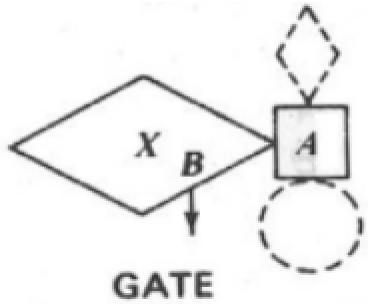
  PRIORITY +,5
  This increases the current priority of the transaction by 5. If the current priority was 10, it will become 15.

- Decreasing the Priority

  PRIORITY -, 2
  This decreases the current priority of the transaction by 2. If the current priority was 10, it will become 8.

15. GATE BLOCK:

A GATE Block alters Transaction flow based on the state of an entity.



Syntax:

GATE O A, B

O - Conditional operator. Condition required of entity to be tested for successful test. Required. The operator must be LS, LR, U, NU, SF, SNF, SE, SNE etc. (LS=Logic Switch Set, LR=Logic Switch Reset, U=Facility in Use, NU=Facility in Not Use, SF=Storage Full, SNF=Storage Not Full, SE=Storage Empty, SNE=Storage Not Empty)

A - Entity name or number to be tested. The entity type is implied by the conditional operator. Required.

B - Destination Block number when test is unsuccessful. Optional.

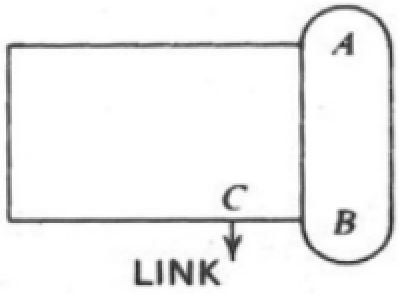
Example:

GATE SNF MotorPool

In this example of a "Refuse Mode" GATE Block, the Active Transaction enters the GATE Block if the Storage Entity named MotorPool is not full (i.e. if at least 1 unit of storage is available). If the Storage is full, the Active Transaction is blocked until 1 or more storage units become available.

16. LINK BLOCK:

In GPSS (General Purpose Simulation System), the LINK block is used to join a transaction to a specified chain (a linked list of transactions). This block is particularly useful for managing queues and other structures where transactions need to be processed in a specific order.



Syntax:
LINK ChainName, Position

Where:

- **ChainName** is the name of the chain to which the transaction is to be linked.
- **Position** specifies where in the chain the transaction should be placed. This can be a specific position in the chain or a relative position (e.g., at the beginning or end of the chain).

Position Values

- HEAD or FIRST: Places the transaction at the beginning of the chain.
- TAIL or LAST: Places the transaction at the end of the chain.
- A specific numeric value indicating the exact position in the chain.

Example:

LINK WAITING_CHAIN, TAIL

Adds the transaction to the end of the chain named WAITING_CHAIN.

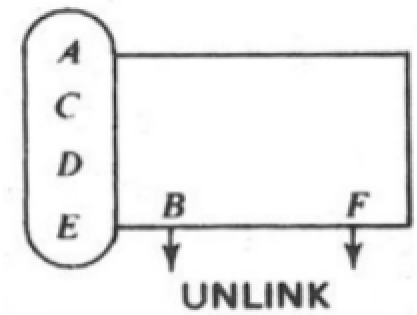
UNLINK WAITING_CHAIN:

Removes the transaction from the chain named WAITING_CHAIN.

Teksan Gharti

17. UNLINK BLOCK:

In GPSS (General Purpose Simulation System), the UNLINK block is used to remove a transaction from a specified chain. This is useful for managing queues and other lists of transactions where transactions need to be processed and removed in a specific order.



Syntax:

UNLINK O A, B, C, D, E, F

O - Relational operator. Relationship of D to E for removal to occur. Optional. The operator must be Null, E, G, GE, L, LE or NE.

A - User Chain number. User Chain from which one or more Transactions will be removed. Required.

B - Block number. The destination Block for removed Transactions. Required.

C - Removal limit. The maximum number of Transactions to be removed. If not specified, ALL is used. Optional.

D - Test value. The member Transaction Parameter name or number to be tested, a Boolean variable to be tested, or BACK to remove from the tail of the chain. Optional.

E - Reference value. The value against which the D Operand is compared. Optional.

F - Block number. The alternate destination for the entering Transaction. Optional.

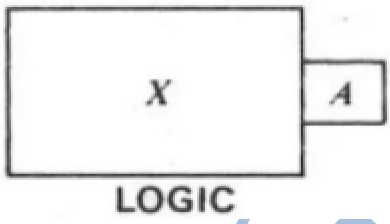
Example:

UNLINK WAITING_CHAIN:

Removes the transaction from the chain named WAITING_CHAIN.

18. LOGIC BLOCK:

The LOGIC block is used to evaluate a logical condition and transfer control to a specified block based on the result of the evaluation. The LOGIC block allows for the implementation of decision-making processes within the simulation model.



Syntax

LOGIC Condition, Label

Where:

- Condition is the logical expression to be evaluated. This can include comparisons and logical operations.
- Label is the label of the block to which control is transferred if the condition is true. If the condition is false, the transaction proceeds to the next block in sequence.

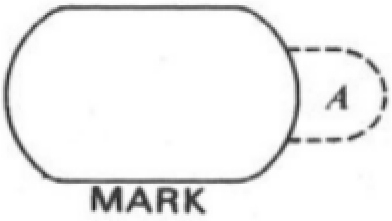
Example:

LOGIC C1 >= 5, PROCESS_MORE:

Evaluates the condition C1 >= 5. If the condition is true, the transaction is transferred to the block labeled PROCESS_MORE. If the condition is false, the transaction proceeds to the next block in sequence.

19. MARK

The MARK block is used to set or update the value of a user-defined attribute for a transaction. This is useful for tracking specific properties or states of a transaction as it moves through the simulation model.



Syntax

MARK AttributeName, Value

Where:

- AttributeName is the name of the attribute to be set or updated.
- Value is the value to be assigned to the attribute. This can be a constant, a variable, or an expression.

Example

MARK PRIORITY, 5:

Sets the attribute PRIORITY of the transaction to 5.

20. TABULATE:

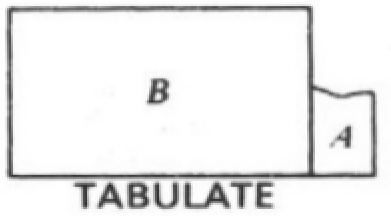
A Table Entity is a set of integers used to accumulate data for a histogram. Each integer represents a frequency class in a histogram. A Table Entity is defined by a TABLE Command.

In another words…

The TABULATE statement is typically used within a GPSS model to collect and analyze various statistics that help in understanding the behavior of the simulated system. It allows simulation analysts to monitor and evaluate performance

metrics, identify bottlenecks, and make informed decisions about system improvements.

TABULATE Blocks update the histogram data accumulated in a Table Entity.



Syntax:

TABULATE A, B

A - Table Entity name or number. Required.

B - Weighting factor. Optional.

Example:

TABULATE Sales

It typically means you are creating a table or report named "Sales" where you will store and analyze specific metrics related to sales transactions or values within your simulation model.

TABULATE Sales, TALLY

Records the number of transactions (sales) in a table named "Sales". The TALLY option counts each transaction and stores the count in the table.

Standard Numerical Attributes (SNAs):

SNAs Contains

- One or two letter code
- A number

For example:

'Storage number 5' is denoted by S5

'Length of queue number 15' is denoted by Q15

SNA's combined with operators can form variable statements

myVariable VARIABLE S6+5*(Q12+Q17)

Similarly, to define floating point, FVARIABLE is used.

Value of SNA's are calculated at the time they are required. The value is never maintained for future use. To save value of SNA, SAVEVALUE is used.

Following table shows the list of some useful SNAs

C1 - Current value of clock

CHn - Number of transactions on chain 'n'

Fn - Current status of facility number 'n'. 1- if facility is busy, 0 otherwise

FNn - Value of function 'n'.

Kn - Integer 'n'

M1 - Transit time of a transaction

Nn - Total number of transactions that have entered block 'n'

Pxn - Parameter number 'n' of transaction, of type 'x'

Qn - Length of queue 'n'

Rn - Space remaining in storage 'n'

RNn - A computed random number having one of the values 1 to 999 with equal probability. 'n' = 1, 2, . . ., 8

Sn - Current occupancy of storage 'n'

Vn - Value of variable statement number 'n'

Wn - Number of transactions currently at block 'n'

Xxn - Value of savevalue location 'n' of type 'x'

Entities:

Transaction entities: GENERATE, SPLIT, TRANSFER, TERMINATE

Facilities entities: SEIZE, RELEASE

Queue entities: QUEUE, DEPART

Storage entities: ENTER, LEAVE

Format of GPSS Program

In GPSS each separate Line of the program will either be a statement or a block. (In a few rare situations the GPSS statement will be continued for two or more Lines). A general format of a GPSS block consists of four separate items. These are:

1. Label or location
2. Operation or block statement
3. Operands
4. Comments

Example-1: A barber shop simulator

We are modeling a barber shop with the following qualities:

- The shop contains one barber and one barber's chair, open for eight hours in a day.
- Customers arrive on average every 18 minutes, with the arrival time varying between 12 and 24 minutes.
- If the barber is busy, the customer will wait in a queue.
- Once the barber is free, the next customer will have a haircut.
- Each haircut takes between 12 and 18 minutes, with the average being 15 minutes.
- Once the haircut is done, the customer will leave the shop.

We want to answer these questions:

- How utilized is the barber through the day?
- How long does the queue get?
- On average, how long does a customer have to wait.

Solution:

Here we want to find:

1. Barber utilization
2. Maximum/average queue length
3. Average customer waiting time

GPSS program :

```
GENERATE 18,6
QUEUE 1
SEIZE 1
DEPART 1
ADVANCE 15,3
RELEASE 1
TERMINATE 0

GENERATE 480
TERMINATE 1

START 1
```

Output:

Untitled Model 1.6.1 - REPORT

```
            GPSS World Simulation Report - Untitled Model 1.6.1


                 Tuesday, February 24, 2026 06:34:05

          START TIME           END TIME  BLOCKS  FACILITIES  STORAGES
               0.000            480.000     9        1          0


 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE            26             0       0
                    2    QUEUE               26             0       0
                    3    SEIZE               26             0       0
                    4    DEPART              26             0       0
                    5    ADVANCE             26             1       0
                    6    RELEASE             25             0       0
                    7    TERMINATE           25             0       0
                    8    GENERATE             1             0       0
                    9    TERMINATE            1             0       0

FACILITY         ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 1                  26    0.810      14.950  1       27    0    0     0       0

QUEUE              MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 1                   1    0     26     19     0.038      0.710      2.636   0

FEC XN   PRI         BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
    27    0         481.836     27      5      6
    28    0         482.707     28      0      1
    29    0         960.000     29      0      8
```

Answer

1. Barber Utilization

$$\text{Utilization} = \frac{\text{Time barber busy}}{\text{Total simulation time}}$$

Since:

- Average arrival rate ≈ 1 customer every 18 minutes
- Average service time = 15 minutes

Teksan Gharti

Traffic intensity (ρ):

$$\rho = \frac{15}{18} = 0.83$$

So barber utilization ≈ **83%**

Meaning:
Barber is busy about 83% of the day.

2. How Long Does Queue Get?

GPSS report shows:

- Maximum queue length
- Average queue length

Since arrival rate is slightly higher than service rate, small queue will form.

Expected:

- Moderate queue build-up
- But system remains stable because arrival time > service time (18 > 15)

3. Average Waiting Time

GPSS calculates automatically from:

Waiting Time=Time of DEPART−Time of QUEUE entry

Since utilization is high (83%), waiting time will exist but not extremely large.

Hence… The system is

Teksan Gharti

- Single server system
- Infinite queue
- First Come First Serve
- Open queueing system

Mathematically similar to:

M/G/1 queueing model.

Flow of One Customer

Arrival → Queue → Seize Barber → Haircut → Release → Leave

Example 2:

A manufacturing shop is turning out parts at rate of one every 5 minutes. As they are finished, the parts go to an inspector, who takes 4±3 minutes to examine each one and rejects about 10% of the parts.

Model 1: Without inspection or wait

GENERATE 5

ADVANCE 4,3

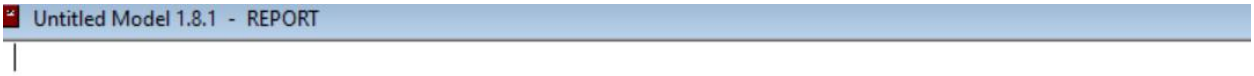
TRANSFER 0.1,REJ,ACC

ACC TERMINATE 1

REJ TERMINATE 1

START 100

Output:



Untitled Model 1.8.1 - REPORT

GPSS World Simulation Report - Untitled Model 1.8.1

Tuesday, February 24, 2026 06:51:18

START TIME	END TIME	BLOCKS	FACILITIES	STORAGES
0.000	501.514	5	0	0

NAME	VALUE
ACC	4.000
REJ	5.000

LABEL	LOC	BLOCK TYPE	ENTRY COUNT	CURRENT COUNT	RETRY
	1	GENERATE	100	0	0
	2	ADVANCE	100	0	0
	3	TRANSFER	100	0	0
ACC	4	TERMINATE	14	0	0
REJ	5	TERMINATE	86	0	0

FEC XN	PRI	BDT	ASSEM	CURRENT	NEXT	PARAMETER	VALUE
101	0	505.000	101	0	1		

Model 2: Wait/With 1 inspection

* Model With 1 Inspector

```
GENERATE 5
QUEUE 1
SEIZE 1
DEPART 1
ADVANCE 4,3

TRANSFER 0.1,REJ,ACC

REJ RELEASE 1
TERMINATE 1

ACC RELEASE 1
TERMINATE 1

START 100
```

Output:

Untitled Model 1.9.1 - REPORT

```
              GPSS World Simulation Report - Untitled Model 1.9.1

                   Tuesday, February 24, 2026 07:00:02

           START TIME            END TIME  BLOCKS  FACILITIES  STORAGES
                0.000             505.946    10         1          0


              NAME                          VALUE
             ACC                             9.000
             REJ                             7.000

 LABEL              LOC   BLOCK TYPE      ENTRY COUNT CURRENT COUNT RETRY
                    1     GENERATE            101              0       0
                    2     QUEUE               101              0       0
                    3     SEIZE               101              1       0
                    4     DEPART              100              0       0
                    5     ADVANCE             100              0       0
                    6     TRANSFER            100              0       0
REJ                 7     RELEASE              92              0       0
                    8     TERMINATE            92              0       0
ACC                 9     RELEASE               8              0       0
                   10     TERMINATE             8              0       0

FACILITY          ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 1                   101    0.755       3.784  1       101    0     0     0       0

QUEUE               MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 1                    1    1    101      61     0.106      0.532      1.342   0
```

Model 2: With 3 inspections

```
GENERATE 5,0
QUEUE 1
SEIZE 1
DEPART 1
ADVANCE 12,9
TRANSFER 0.1,REJ,ACC

REJ RELEASE 1
TERMINATE 1

ACC RELEASE 1
TERMINATE 1

START 100
```


Output:

Untitled Model 1.11.1 - REPORT

```
              GPSS World Simulation Report - Untitled Model 1.11.1


                   Tuesday, February 24, 2026 07:25:42

           START TIME            END TIME  BLOCKS  FACILITIES  STORAGES
                0.000           1195.300    10         1          0


              NAME                          VALUE
          ACC                               9.000
          REJ                               7.000


 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE            239             0       0
                    2    QUEUE               239           138       0
                    3    SEIZE               101             1       0
                    4    DEPART              100             0       0
                    5    ADVANCE             100             0       0
                    6    TRANSFER            100             0       0
REJ                 7    RELEASE              89             0       0
                    8    TERMINATE            89             0       0
ACC                 9    RELEASE              11             0       0
                   10    TERMINATE            11             0       0


FACILITY          ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 1                   101    0.996      11.785  1      101    0     0     0    138


QUEUE               MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 1                  139  139    239       1    67.841    339.290    340.715   0
```

Controlling the Length of a Simulation Run (Time Driven):

The TERMINATE BLOCK can contain an attribute value that will make the simulation stop when the Number of transactions entering the TERMINATE block equals the attribute value.

In a GENERATE statement it is possible to specify the maximum Number of transactions that the GENERATE block will produce. Clearly the model will cease to function once this limit has been reached and the last transaction has been terminated.

It is possible to have a separate timing section in your program that will determine the exact time of a simulation.

For Example:

```
TIMER GENERATE 2000

TERMINATE 1
```

This will cause the GENERATE block to produce a single transaction at 2000 time units after the simulation starts and decrement the termination counter to zero. This will immediately stop the simulation run. You must ensure there is no attribute value in any other TERMINATE block in the simulation model.

Example 1: Joe's Barbershop (Time Driven)

Suppose we want to simulate a typical day of nine hours' work. Since we have decided that one time unit is equal to one minute then we must convert nine hours in to minutes i.e. 9x60 = 540 minutes.

```
MEN     GENERATE    18, 6
        QUEUE       SEAT
        SEIZE       JOE
        DEPART      SEAT
        ADVANCE     15, 3
        RELEASE     JOE
        TERMINATE
```

```
TIMER     GENERATE     540
          TERMINATE    1
          START        1
```

Note: the first TERMINATE block does not have an attribute.

In the program above, after the command START 1 is issued, the simulation begins to run. Due to the fact that the first TERMINATE block does not have an attribute, the termination counter is not decremented and the simulation continues. At 540 time units after the simulation starts, however, a single transaction will be generated by generator which will decrement the termination counter to zero and stop the program.

Example 2: Joe's Barbershop (Event Driven)

Suppose we want to simulate the Barbershop for 50 Transactions.

```
MEN       GENERATE     18, 6
          QUEUE        SEAT
          SEIZE        JOE
          DEPART       SEAT
          ADVANCE      15, 3
          RELEASE      JOE
          TERMINATE    1
          START        50
```

Complete Example with Context

Here is how a QUEUE block might be used in a simple GPSS model:

```
10 GENERATE 5, 2
```

```
20 QUEUE Garage
30 SEIZE Mechanic
40 DEPART Garage
50 ADVANCE 15, 3
60 RELEASE Mechanic
70 TERMINATE 1
```

In this complete example:

- **Line 10**: GENERATE 5, 2 creates transactions with inter-arrival times uniformly distributed between 3 and 7 time units.
- **Line 20**: QUEUE Garage places each entity into the Garage queue.
- **Line 30**: SEIZE Mechanic attempts to seize a resource named Mechanic. If the Mechanic is busy, the entity remains in the Garage queue.
- **Line 40**: DEPART Garage removes the entity from the Garage queue once the Mechanic is seized.
- **Line 50**: ADVANCE 15, 3 delays the entity for a service time uniformly distributed between 12 and 18 time units.
- **Line 60**: RELEASE Mechanic frees the Mechanic resource, making it available for the next entity in the queue.
- **Line 70**: TERMINATE 1 removes the entity from the system.

End of Unit-8